

OS Lab

PRACTICAL-8

Develop a program using malloc(), calloc(), realloc(), and free(). Monitor memory allocation behavior and identify memory leaks using Valgrind.

Develop a program that demonstrates Copy-on-Write behavior after a fork(). Analyze memory usage before and after modifying data in the child process.

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int i;

    // 1. malloc()
    int *a = (int *)malloc(5 * sizeof(int));

    if (a == NULL) {
        printf("malloc failed\n");
        return 1;
    }

    for (i = 0; i < 5; i++) {
        a[i] = (i + 1) * 10;
    }

    printf("Memory allocated using malloc:\n");
    for (i = 0; i < 5; i++) {
        printf("%d ", a[i]);
    }
    printf("\n");

    // 2. calloc()
    int *b = (int *)calloc(5, sizeof(int));

    if (b == NULL) {
        printf("calloc failed\n");
        free(a);
        return 1;
    }
```

```

printf("Memory allocated using calloc:\n");
for (i = 0; i < 5; i++) {
    printf("%d ", b[i]);
}
printf("\n");

// 3. realloc()
a = (int *)realloc(a, 10 * sizeof(int));

if (a == NULL) {
    printf("realloc failed\n");
    free(b);
    return 1;
}

for (i = 5; i < 10; i++) {
    a[i] = (i + 1) * 10;
}

printf("After realloc:\n");
for (i = 0; i < 10; i++) {
    printf("%d ", a[i]);
}
printf("\n");

// 4. free()
free(a);
free(b);

printf("Memory freed successfully.\n");

return 0;
}

```

```

lahari@DESKTOP-AIC33F7:~$ cd ~/OSLab
lahari@DESKTOP-AIC33F7:~/OSLab$ nano memory.c
lahari@DESKTOP-AIC33F7:~/OSLab$ gcc memory.c -o memory
lahari@DESKTOP-AIC33F7:~/OSLab$ ./memory
Memory allocated using malloc:
10 20 30 40 50
Memory allocated using calloc:
0 0 0 0 0
After realloc:
10 20 30 40 50 60 70 80 90 100
Memory freed successfully.

```

```

Memory freed successfully.
lahari@DESKTOP-AIC33F7:~/OSLab$ valgrind --leak-check=full ./memory
==514== Memcheck, a memory error detector
==514== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==514== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==514== Command: ./memory
==514==
Memory allocated using malloc:
10 20 30 40 50
Memory allocated using calloc:
0 0 0 0 0
After realloc:
10 20 30 40 50 60 70 80 90 100
Memory freed successfully.
==514==
==514== HEAP SUMMARY:
==514==   in use at exit: 0 bytes in 0 blocks
==514== total heap usage: 4 allocs, 4 frees, 1,104 bytes allocated
==514==
==514== All heap blocks were freed -- no leaks are possible
==514==
==514== For lists of detected and suppressed errors, rerun with: -s
==514== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)

```

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

```

```

#define SIZE 1000000

```

```

int main() {
    int *data = malloc(SIZE * sizeof(int));

    if (data == NULL) {
        perror("malloc");
        return 1;
    }

    // Initialize memory
    for (int i = 0; i < SIZE; i++) {
        data[i] = i;
    }

    printf("Before fork:\n");
    printf("Parent PID: %d\n", getpid());
    printf("Address of data: %p\n", (void *)data);

    pid_t pid = fork();

```

```

if (pid < 0) {
    perror("fork");
    free(data);
    return 1;
}

if (pid == 0) {
    // Child process
    printf("\nChild process:\n");
    printf("Child PID: %d\n", getpid());
    printf("Address of data: %p\n", (void *)data);

    printf("Before modification: data[0] = %d\n", data[0]);

    // Modify memory
    data[0] = 9999;

    printf("After modification: data[0] = %d\n", data[0]);

    sleep(5);

    free(data);
    exit(0);
}

else {
    // Parent process
    printf("\nParent process:\n");
    printf("Parent PID: %d\n", getpid());

    sleep(2);

    printf("Parent data[0] = %d\n", data[0]);

    wait(NULL);

    printf("\nChild process completed.\n");
    printf("Parent data[0] is still = %d\n", data[0]);

    free(data);
}

return 0;

```

}

```
lahari@DESKTOP-AIC33F7:~/OSLab$ nano memory.c
lahari@DESKTOP-AIC33F7:~/OSLab$ gcc memory.c -o memory
lahari@DESKTOP-AIC33F7:~/OSLab$ valgrind --leak-check=full ./memory
==527== Memcheck, a memory error detector
==527== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==527== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==527== Command: ./memory
==527==
Memory allocated using malloc:
10 20 30 40 50
Memory allocated using calloc:
0 0 0 0 0
After realloc:
10 20 30 40 50 60 70 80 90 100
Memory freed successfully.
==527==
==527== HEAP SUMMARY:
==527==   in use at exit: 20 bytes in 1 blocks
==527==   total heap usage: 4 allocs, 3 frees, 1,104 bytes allocated
==527==
==527== 20 bytes in 1 blocks are definitely lost in loss record 1 of 1
==527==    at 0x484D953: calloc (in /usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
==527==    by 0x1092E1: main (in /home/lahari/OSLab/memory)
==527==
==527== LEAK SUMMARY:
==527==   definitely lost: 20 bytes in 1 blocks
==527==   indirectly lost: 0 bytes in 0 blocks
==527==   possibly lost: 0 bytes in 0 blocks
==527==   still reachable: 0 bytes in 0 blocks
==527==   suppressed: 0 bytes in 0 blocks
==527==
==527== For lists of detected and suppressed errors, rerun with: -s
==527== ERROR SUMMARY: 1 errors from 1 contexts (suppressed: 0 from 0)
```

```
lahari@DESKTOP-AIC33F7:~/OSLab$ nano cow.c
lahari@DESKTOP-AIC33F7:~/OSLab$ gcc cow.c -o cow
lahari@DESKTOP-AIC33F7:~/OSLab$ ./cow
Before fork:
Parent PID: 540
Address of data: 0x780b1882f010

Parent process:
Parent PID: 540

Child process:
Child PID: 541
Address of data: 0x780b1882f010
Before modification: data[0] = 0
After modification: data[0] = 9999
Parent data[0] = 0

Child process completed.
Parent data[0] is still = 0
lahari@DESKTOP-AIC33F7:~/OSLab$ ps -ef | grep cow
lahari      543      317  0 06:59 pts/0    00:00:00 grep --color=auto cow
```